



University
Mohammed VI
Polytechnic



Deliverable #6: Lab 6; Views, Triggers, and Application development.

Data Management Course
UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering
Session: Fall 2025

Team Information

Team Name	
Member 1	Adam Chikri
Member 2	Amine Abdellaoui
Member 3	Douae Bouayadi
Member 4	Aya Abid
Member 5	Nissrine Benallal
Member 6	Yasmine Bouaboud
Repository Link	https://github.com/Adam-chi/Lab6-DBMS

1 Introduction

This Lab 6 deliverable implements views, triggers, and a Python application for the MNHS database system. Views are created to simplify complex queries for appointment scheduling, medication pricing, staff workload analysis, and patient visit tracking. Triggers enforce critical rules including staff double booking prevention, automatic expense calculation for prescriptions, stock validation, and patient data integrity protection. A python application provides practical interfaces for common database operations including patient management, appointment scheduling, inventory monitoring, and staff workload analysis. This work demonstrates how database features can automate logic and maintain data consistency without requiring manual intervention.

2 Requirements

SQL Views:

The first part of the lab required us to create four SQL views that summarize key information from the MNHS database. These views include upcoming scheduled appointments per hospital, drug price statistics, staff workload over the last thirty days, and each patient's next scheduled visit. The goal was to simplify complex queries and provide reusable structures that support data analysis and reporting.

SQL Triggers:

The second section focused on implementing triggers to enforce important business rules and maintain database integrity. We were required to prevent double-booking of staff, automatically recalculate prescription expenses, validate medication stock and pricing changes, and block the deletion of patients who still have clinical activities. Each trigger had to include an example demonstrating that it works correctly.

Backend Application:

The final part involved developing a small backend application connected to the MNHS MySQL database. It had to implement four commands: listing the first twenty patients by last name, scheduling a new appointment with its clinical activity in a transaction, identifying low-stock medications per hospital, and calculating each staff member's share of appointments. The application needed to follow good practices such as parameterized queries, safe error handling, and clear modular design.

3 Methodology

SQL Views:

View 1:

To build this view, we joined Appointment → ClinicalActivity → Department → Hospital to correctly associate each appointment with its hospital. We filtered only rows with Status = 'Scheduled' and limited the dates to the next fourteen days. Grouping by hospital and date allowed us to count scheduled appointments per day, per hospital. We chose this SQL-side aggregation to avoid repeating logic in the backend. The view provides a clean, reusable summary for planning needs. Letting the application run a simple SELECT instead of rebuilding the complex logic each time.

View 2:

We create (or replace) the view named MedicationPricingSummary, needing to join Stock, Hospital, Medication tables. So that we can get the minimum, maximum, and average price of a medication in a hospital (that's why we group by medication and hospital) giving us the desired results. This view centralizes all price statistics per hospital-medication pair, so the application can retrieve averages, minimum, maximum, and timestamps with a single query rather than recalculating aggregations or joining three tables repeatedly. Leaving

View 3:

To build this view, we joined Staff → ClinicalActivity → Appointment to associate staff members with their appointments while preserving all staff records through LEFT JOINS. We filtered for activities from the last 30 days and used conditional aggregation with SUM(IF(...)) to count appointments by status (Scheduled, Completed, Cancelled) in a single query. This server-side aggregation eliminates the need for multiple backend queries and provides a ready-to-use workload summary for reporting and dashboard features. The StaffWorkloadThirty view simplifies application code by abstracting the underlying complex joins and conditional aggregations required to calculate staff workload metrics. This encapsulation provides a consistent and reusable data interface, improving both code maintainability and query performance by pre-defining the logic in the database layer.

View 4:

The PatientNextVisit view is designed to simplify access to each patient's upcoming scheduled visit. It joins the Patient, ClinicalActivity, Appointment, Department, and Hospital tables to retrieve, for each patient, their next appointment date, department name, hospital name, and city. The view filters appointments to include only those with status 'Scheduled' and dates strictly after the current date, and it selects the earliest such date using MIN(C.Date).

Encapsulating these joins and filters inside the PatientNextVisit view greatly simplifies application code, since the next scheduled visit for any patient can be retrieved with a

single SELECT rather than reconstructing the logic each time. It also improves performance by allowing the database to optimize the underlying query once and reuse it efficiently.

SQL Triggers:

Trigger 1:

To build this trigger we referred to an IF statement to check if the inserting or updating could create a double booking (same date and time) for the same staff member.

The first step is getting the Date, Time and Staff_member from the ClinicalActivity table by declaring new variables that store the information of the clinical activity that is going to be inserted or updated in the Appointment table (because inserting or updating in the Appointment table requires a row of (CAID, IID, STAFF_ID, DEP_ID, Date, Time) in the ClinicalActivity table).

The second step is checking if another appointment exists with the same staff member, date and time using the EXISTS() function, where we search for any appointment that has the same date, time and staff member (related to the clinical activity to insert or update in Appointment) but with a different CAID.

Trigger 2:

In this Trigger 2 implementation, the purpose is to ensure that the total expense associated with a clinical activity is always correct whenever prescription lines change. This is why three triggers are needed: AFTER INSERT, AFTER UPDATE, and AFTER DELETE, since each of these operations can modify the medications included in a prescription. Inside each trigger, new_total stores the recalculated sum of medication prices, while missing_price checks whether any medication has a missing or invalid price in Stock. A LEFT JOIN is used during the validation step to detect medications that are not found in stock (producing NULL), whereas regular joins are used for the actual price calculation. If at least one price is missing, the trigger raises an exception using SIGNAL SQLSTATE '45000', preventing inconsistent data. After validation, the new total is computed and updated in Expense through the link with Prescription and ClinicalActivity. The use of IFNULL(new_total, 0) ensures clean results when no items remain in a prescription, avoiding NULL values and keeping the database consistent.

Trigger 3:

To implement this trigger, IF statements are used to verify that all value constraints on Qty, UnitPrice, and ReorderLevel are respected. For insert operations, the trigger must reject any attempt to insert a stock item where Qty is less than zero, UnitPrice is less than or equal to zero, or ReorderLevel is negative. Similarly, during update operations, the trigger must reject any modification that results in invalid values for these same fields. In addition to these constraints, the trigger must also ensure that any decrease in quantity does not cause the available Qty to drop below zero, preventing inconsistencies in stock levels.

Trigger 4:

Trigger 4 ensures that a patient cannot be deleted from the database if they still have clinical activities linked to them. It is defined as a BEFORE DELETE trigger so that the deletion is checked and blocked before it happens. Inside the trigger, the variable `activity_count` is declared to store the number of clinical activities associated with the patient being removed.

A simple `SELECT COUNT(*) INTO activity_count` retrieves this number using the patient's identifier `OLD.IID`. If the count is greater than zero, the trigger raises an error using `SIGNAL SQLSTATE '45000'`, which stops the deletion and returns a clear message instructing the user to reassign or delete the dependent activities first. This mechanism protects referential integrity and prevents orphaned records in the `ClinicalActivity` table.

App Development:

Task 1:

For this command, we used a Python function that queries the first twenty patients ordered by last name. Since the database stores the full name in one field, we used `SUBSTRING.INDEX` to extract the last name for sorting. We used a dictionary cursor and context managers to keep the code safe and readable. The function only retrieves data, while the main program prints it. This modular approach matches the lab structure and keeps the backend easy to extend.

An easy way to access this method for users through the cmd, using:

```
python main.py list_patients
```

Task 2:

An efficient appointment-scheduling system was added through the `schedule_appointment` function in Python. This function builds the SQL queries automatically from the arguments provided when it is called, and creates both the corresponding `ClinicalActivity` and `Appointment` entries using the supplied data. It then executes these queries within a try and catch block to detect any errors and report them if the creation fails.

To make the feature even more accessible, argument parsing was introduced. Users can now schedule an appointment directly from the command line by running a single command with the required parameters, for example:

```
python main.py schedule_appt --caid ... --iid ... --staff ... --dep ...  
--date ... --time ... --reason ...
```

Task 3:

For this command, we implemented a Python function that queries the database to identify medications requiring restocking, as well as the medications that are not within that hospital. The idea is to treat every hospital-medication pair as something that should exist, even if the hospital doesn't actually stock that medication. To do that, you first generate all possible combinations of hospitals and medications, then attach stock information when it exists and leave it empty when it doesn't. Once you have this full table, you simply filter for the problematic cases: either the hospital has the medication but its quantity is below the reorder level, or the hospital doesn't have any stock entry at all. This way, a medication that a hospital doesn't stock still appears in the results, because the absence of stock is itself considered a problem. The result is then returned as a list containing dictionaries with the wanted data.

To access this feature, the user must run the command on cmd: `python main.py low_stock`

Task 4:

The function `staff_share` executes a SQL query that joins the `Staff`, `ClinicalActivity`, `Appointment`, and `Department` tables to aggregate the total appointments per staff member. Additionally, it computes the percentage share by comparing each staff member's total appointments to the total number of appointments in their hospital. Finally, the results are returned as a list of dictionaries, allowing the application to access this information efficiently without having to reconstruct the complex query logic.

For a user to access the feature, he must run the cmd: `python main.py staff_share`

4 Implementation & Results

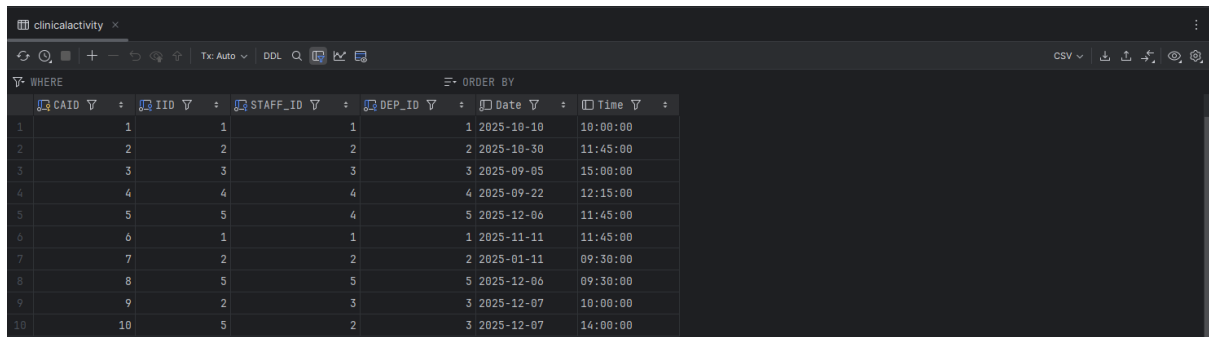
For the sake of testing, the data inserted, updated into the database doesn't follow any logic whatsoever and might make no sense, it's only to show that the queries match the results.

SQL Views:

View 1:

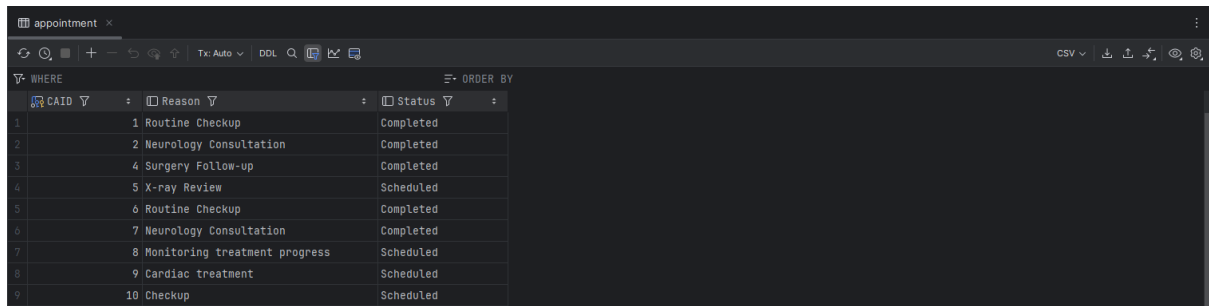
Consider the following tables with the following data:

Clinical Activity:



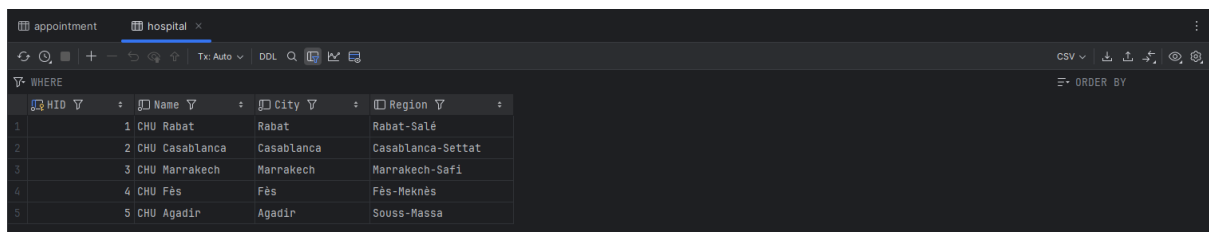
	CAID	IID	STAFF_ID	DEP_ID	Date	Time
1		1		1	2025-10-10	10:00:00
2		2		2	2025-10-30	11:45:00
3		3		3	2025-09-05	15:00:00
4		4		4	2025-09-22	12:15:00
5		5		4	2025-12-06	11:45:00
6		6	1	1	2025-11-11	11:45:00
7		7	2	2	2025-01-11	09:30:00
8		8	5	5	2025-12-06	09:30:00
9		9	2	3	2025-12-07	10:00:00
10		10	5	2	2025-12-07	14:00:00

Appointment:



	CAID	Reason	Status
1		Routine Checkup	Completed
2		Neurology Consultation	Completed
3		Surgery Follow-up	Completed
4		X-ray Review	Scheduled
5		Routine Checkup	Completed
6		Neurology Consultation	Completed
7		Monitoring treatment progress	Scheduled
8		Cardiac treatment	Scheduled
9		Checkup	Scheduled
10			

Hospital:



	HID	Name	City	Region
1		CHU Rabat	Rabat	Rabat-Salé
2		CHU Casablanca	Casablanca	Casablanca-Settat
3		CHU Marrakech	Marrakech	Marrakech-Safi
4		CHU Fès	Fès	Fès-Meknès
5		CHU Agadir	Agadir	Souss-Massa

Department:

DEP_ID	HID	Name	Specialty
1	1	Cardiology	Heart Diseases
2	1	Neurology	Brain and Nerves
3	2	Pediatrics	Child Health
4	3	Surgery	General Surgery
5	4	Radiology	Imaging

Running the following query to create the view:

```

1 CREATE OR REPLACE VIEW UpcomingByHospital AS
2 SELECT
3     H.Name AS HospitalName ,
4     CA.Date AS ApptDate ,
5     COUNT(*) AS ScheduledCount
6 FROM Appointment A
7 JOIN ClinicalActivity CA
8     ON A.CAID = CA.CAID
9 JOIN Department D
10    ON CA.DEP_ID = D.DEP_ID
11 JOIN Hospital H
12    ON D.HID = H.HID
13 WHERE
14     A.Status = 'Scheduled'
15     AND CA.Date BETWEEN CURDATE() AND DATE_ADD(CURDATE(),
16         INTERVAL 14 DAY)
17 GROUP BY
18     H.Name , CA.Date;
```

This create the following view with the following data:

HospitalName	ApptDate	ScheduledCount
CHU Fès	2025-12-06	2
CHU Casablanca	2025-12-07	2

View 2:

Consider the following tables with the following data:

Stock:

HID	MID	StockTimestamp	UnitPrice	Qty	ReorderLevel
1	1	2025-11-16 04:34:03	10.00	100	10
1	1	2025-11-28 05:00:00	30.00	100	10
1	3	2025-11-20 04:00:00	20.00	50	10
2	2	2025-11-16 04:34:03	25.00	50	5
2	3	2025-11-16 04:34:03	7.00	200	20
3	3	2025-11-16 04:34:03	5.00	200	20
3	4	2025-11-16 04:34:03	10.00	200	20
4	4	2025-11-16 04:34:03	15.00	30	10
5	5	2025-11-16 04:34:03	50.00	10	5

When running the query to create the view:


```

1 CREATE OR REPLACE VIEW MedicationPricingSummary AS
2 SELECT
3     s.HID,
4     h.Name as HospitalName,
5     s.MID,
6     m.Name as MedicationName,
7     AVG(s.UnitPrice) AS AvgUnitPrice,
8     MIN(s.UnitPrice) AS MinUnitPrice,
9     MAX(s.UnitPrice) AS MaxUnitPrice,
10    MAX(s.StockTimestamp) AS LastStockTimestamp
11 FROM Stock s
12 JOIN Hospital h ON s.HID = h.HID
13 JOIN Medication m ON s.MID = m.MID
14 GROUP BY s.HID, h.Name, s.MID, m.Name;

```

The following image is the result of the view which aligns perfectly with what is asked:

HID	HospitalName	MID	MedicationName	AvgUnitPrice	MinUnitPrice	MaxUnitPrice	LastStockTimestamp
1	CHU Rabat	1	Paracetamol	20.000000	10.00	30.00	2025-11-28 05:00:00
1	CHU Rabat	3	Aspirin	20.000000	20.00	20.00	2025-11-20 04:00:00
2	CHU Casablanca	2	Amoxicillin	25.000000	25.00	25.00	2025-11-10 04:34:03
2	CHU Casablanca	3	Aspirin	7.000000	7.00	7.00	2025-11-10 04:34:03
3	CHU Marrakech	3	Aspirin	5.000000	5.00	5.00	2025-11-10 04:34:03
3	CHU Marrakech	4	Cough Syrup	10.000000	10.00	10.00	2025-11-10 04:34:03
4	CHU Fes	4	Cough Syrup	15.000000	15.00	15.00	2025-11-10 04:34:03
5	CHU Agadir	5	Insulin	50.000000	50.00	50.00	2025-11-10 04:34:03

View 3:

Consider the following tables:

Staff:

STAFF_ID	FullName	Status
1	Dr. Hicham El Idrissi	Active
2	Dr. Salma Tazi	Active
3	Nurse Rania Lahlou	Active
4	Technician Yassine Kabbaj	Retired
5	Dr. Mehdi Rahali	Active

ClinicalActivity:

CAID	IID	STAFF_ID	DEP_ID	Date	Time
1	1	1	1	2025-10-10	10:00:00
2	2	2	2	2025-10-30	11:45:00
3	3	3	3	2025-09-05	15:00:00
4	4	4	4	2025-09-22	12:15:00
5	5	5	4	2025-12-06	11:45:00
6	6	1	1	2025-11-11	11:45:00
7	7	2	2	2025-01-11	09:30:00
8	8	5	5	2025-12-06	09:30:00
9	9	2	3	2025-12-07	10:00:00
10	10	5	2	2025-12-07	14:00:00

Appointment:

CAID	Reason	Status
1	Routine Checkup	Completed
2	Neurology Consultation	Completed
3	Surgery Follow-up	Completed
4	X-ray Review	Scheduled
5	Routine Checkup	Completed
6	Neurology Consultation	Completed
7	Monitoring treatment progress	Scheduled
8	Cardiac treatment	Scheduled
9	Checkup	Scheduled

```

1 CREATE OR REPLACE VIEW StaffWorkloadThirty AS
2 SELECT
3     S.STAFF_ID ,
4     S.FullName ,
5     COUNT(A.CAID) AS TotalAppointments ,
6     SUM(IF(A.Status = 'Scheduled', 1, 0)) AS ScheduledCount ,
7     SUM(IF(A.Status = 'Completed', 1, 0)) AS CompletedCount ,
8     SUM(IF(A.Status = 'Cancelled', 1, 0)) AS CancelledCount
9 FROM Staff S
10 LEFT JOIN ClinicalActivity CA ON S.STAFF_ID = CA.STAFF_ID
11 LEFT JOIN Appointment A ON CA.CAID = A.CAID
12 WHERE CA.Date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
13        OR CA.Date IS NULL
14 GROUP BY S.STAFF_ID , S.FullName;

```

Following image is the produced result of running the query:

STAFF_ID	FullName	TotalAppointments	ScheduledCount	CompletedCount	CancelledCount
1	Dr. Hicham El Idrissi	1	0	1	0
2	Dr. Salma Tazi	2	1	1	0
3	Nurse Rania Lahlou	1	1	0	0
4	Technician Yassine Kabbaj	1	1	0	0
5	Dr. Mehdi Rahali	1	1	0	0

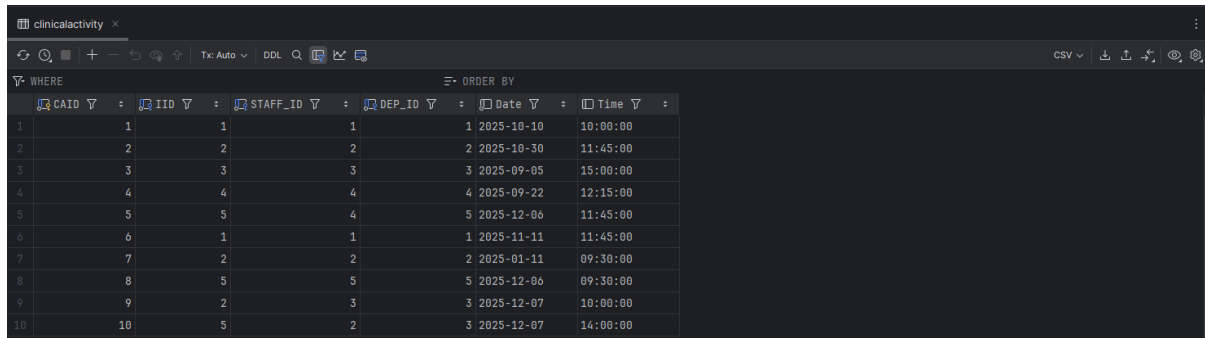
View 4:

Consider the following tables:

Patient:

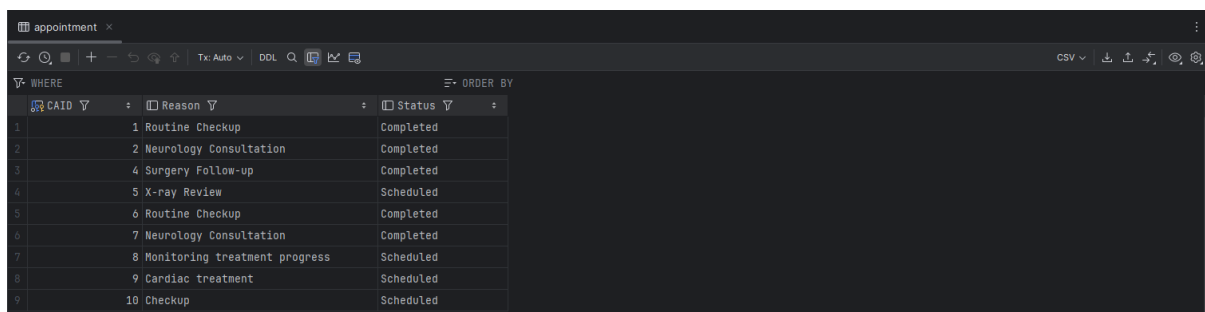
ID	CIN	FullName	Birth	Sex	BloodGroup	Phone	Email
1	CIN1001	Sara El Amrani	1995-06-21	F	A+	0612345678	<null>
2	CIN1002	Omar Bennani	1988-09-15	M	O-	0623456789	<null>
3	CIN1003	Youssef Khadir	2000-01-10	M	B+	0611223344	<null>
4	CIN1004	Lina Berrada	1993-11-05	F	AB-	0645678901	<null>
5	CIN1005	Amine Fassi	1998-04-20	M	A-	0656789012	<null>

ClinicalActivity:



CAID	IID	STAFF_ID	DEP_ID	Date	Time
1	1	1	1	2025-10-10	10:00:00
2	2	2	2	2025-10-30	11:45:00
3	3	3	3	2025-09-05	15:00:00
4	4	4	4	2025-09-22	12:15:00
5	5	5	4	2025-12-06	11:45:00
6	6	1	1	2025-11-11	11:45:00
7	7	2	2	2025-01-11	09:30:00
8	8	5	5	2025-12-06	09:30:00
9	9	2	3	2025-12-07	10:00:00
10	10	5	2	2025-12-07	14:00:00

Appointment:



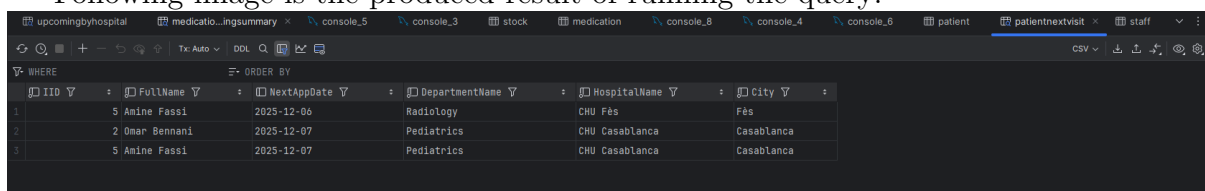
CAID	Reason	Status
1	1 Routine Checkup	Completed
2	2 Neurology Consultation	Completed
3	4 Surgery Follow-up	Completed
4	5 X-ray Review	Scheduled
5	6 Routine Checkup	Completed
6	7 Neurology Consultation	Completed
7	8 Monitoring treatment progress	Scheduled
8	9 Cardiac treatment	Scheduled
9	10 Checkup	Scheduled

```

1 CREATE OR REPLACE VIEW PatientNextVisit AS
2 SELECT P.IID,P.FullName,MIN(C.Date) AS NextAppDate,D.Name AS
   DepartmentName,H.Name AS HospitalName,H.City
3 FROM Patient P
4 JOIN ClinicalActivity C on P.IID=C.IID
5 JOIN Appointment A on A.CAID=C.CAID
6 JOIN Department D on D.DEP_ID=C.DEP_ID
7 JOIN Hospital H on H.HID=D.HID
8 WHERE A.status='Scheduled' AND C.Date>CURDATE()
9 GROUP BY P.IID,D.Name,H.Name,H.City;

```

Following image is the produced result of running the query:



IID	FullName	NextAppDate	DepartmentName	HospitalName	City
5	Amine Fassi	2025-12-06	Radiology	CHU Fès	Fès
2	Omar Benmani	2025-12-07	Pediatrics	CHU Casablanca	Casablanca
5	Amine Fassi	2025-12-07	Pediatrics	CHU Casablanca	Casablanca

It wasn't clearly explained whether the result should be per hospital or across all hospitals, so we decided to implement a version where it's per hospital thus showing the patient with iid 5 twice since he has different appointments in different hospitals

SQL Triggers:

Trigger 1:

```

1 DELIMITER $$
2
3 CREATE TRIGGER Reject_Double_Update
4 BEFORE UPDATE ON appointment
5 FOR EACH ROW
6 BEGIN
7
8     DECLARE newDate DATE;
9     DECLARE newTime TIME;
10    DECLARE newStaff INT;
11
12    SELECT Date, Time, STAFF_ID
13    INTO newDate, newTime, newStaff
14    FROM clinicalactivity
15    WHERE CAID = NEW.CAID;
16
17
18    IF EXISTS (
19        SELECT 1
20        FROM clinicalactivity C
21        JOIN appointment A ON C.CAID = A.CAID
22        WHERE C.Date = newDate
23              AND C.Time = newTime
24              AND C.STAFF_ID = newStaff
25              AND A.CAID <> NEW.CAID
26    ) THEN
27        SIGNAL SQLSTATE '45000'
28        SET MESSAGE_TEXT = 'This appointment conflicts with
29                          an existing one.';
30    END IF;
31
32    END$$
33
34 DELIMITER ;
35
36 CREATE TRIGGER Reject_Double_Insert
37 BEFORE INSERT ON appointment
38 FOR EACH ROW
39 BEGIN
40
41     DECLARE newDate DATE;
42     DECLARE newTime TIME;
43     DECLARE newStaff INT;
44
45     SELECT Date, Time, STAFF_ID
46     INTO newDate, newTime, newStaff
47     FROM clinicalactivity
48     WHERE CAID = NEW.CAID;

```

```

47
48
49 IF EXISTS (
50     SELECT 1
51     FROM clinicalactivity C
52     JOIN appointment A ON C.CAID = A.CAID
53     WHERE C.Date = newDate
54           AND C.Time = newTime
55           AND C.STAFF_ID = newStaff
56           AND A.CAID <> NEW.CAID
57 ) THEN
58     SIGNAL SQLSTATE '45000'
59     SET MESSAGE_TEXT = 'This appointment conflicts with
60                        an existing one.';
61 END IF;
62 END$$
DELIMITER ;

```

And consider the following ClinicalActivity table:

CAID	IID	STAFF_ID	DEP_ID	Date	Time
1	1	1	1	2025-10-10	10:00:00
2	2	2	2	2025-10-30	11:45:00
3	3	3	3	2025-09-05	15:00:00
4	4	4	4	2025-09-22	12:15:00
5	5	5	4	2025-12-06	11:45:00
6	6	1	1	2025-11-11	11:45:00
7	7	2	2	2025-01-11	09:30:00
8	8	5	5	2025-12-06	09:30:00
9	9	2	3	2025-12-07	10:00:00
10	10	5	2	2025-12-07	14:00:00
11	11	1	2	2025-12-07	14:00:00

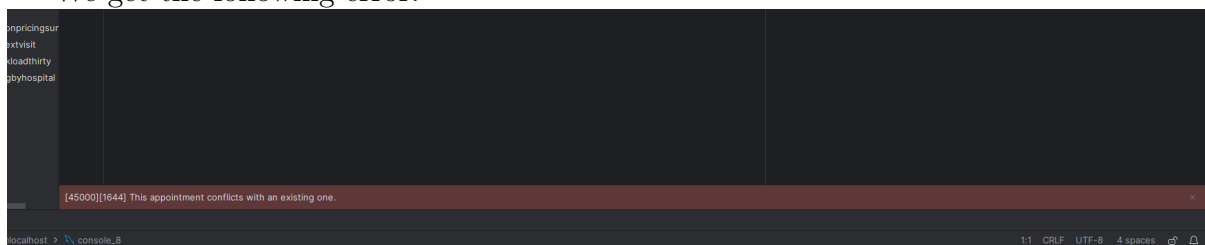
When running the following query:

```

1 INSERT INTO Appointment (CAID, Reason, Status)
2 VALUES (11, 'Follow-up', 'Scheduled');

```

We get the following error:



Similarly, when trying to update:

```

1 UPDATE appointment SET CAID = 11 WHERE CAID = 9;

```

We get the following error:



Trigger 2:

We create the trigger with the following query:

```

1 DELIMITER //
2 CREATE TRIGGER RecomputeExpenseAfterInsert
3 AFTER INSERT ON Includes
4 FOR EACH ROW
5 BEGIN
6     DECLARE new_total DECIMAL(10,2);
7     DECLARE missing_price INT;
8
9     SELECT COUNT(*) INTO missing_price
10    FROM Includes I
11    JOIN Prescription PR ON I.PID = PR.PID
12    JOIN ClinicalActivity CA ON PR.CAID = CA.CAID
13    JOIN Department D ON CA.DEP_ID = D.DEP_ID
14    LEFT JOIN Stock S ON S.MID = I.MID AND S.HID = D.HID
15    WHERE I.PID = NEW.PID AND (S.UnitPrice IS NULL OR
16        S.UnitPrice <= 0);
17
18    IF missing_price > 0 THEN
19        SIGNAL SQLSTATE '45000'
20        SET MESSAGE_TEXT = 'Error: Cannot compute expense, medication price is missing or invalid in stock.';
21    END IF;
22
23    SELECT SUM(S.UnitPrice) INTO new_total
24    FROM Includes I
25    JOIN Prescription PR ON I.PID = PR.PID
26    JOIN ClinicalActivity CA ON PR.CAID = CA.CAID
27    JOIN Department D ON CA.DEP_ID = D.DEP_ID
28    JOIN Stock S ON S.MID = I.MID AND S.HID = D.HID
29    WHERE I.PID = NEW.PID;
30
31    UPDATE Expense E
32    JOIN Prescription PR ON E.CAID = PR.CAID
33    SET E.Total = IFNULL(new_total, 0)
34    WHERE PR.PID = NEW.PID;
35
36 DELIMITER ;
37
38 DELIMITER //
39

```

```

40 CREATE TRIGGER RecomputeExpenseAfterUpdate
41 AFTER UPDATE ON Includes
42 FOR EACH ROW
43 BEGIN
44
45     DECLARE new_total DECIMAL(10,2);
46     DECLARE missing_price INT;
47
48     SELECT COUNT(*) INTO missing_price
49     FROM Includes I
50     JOIN Prescription PR ON I.PID = PR.PID
51     JOIN ClinicalActivity CA ON PR.CAID = CA.CAID
52     JOIN Department D ON CA.DEP_ID = D.DEP_ID
53     LEFT JOIN Stock S ON S.MID = I.MID AND S.HID = D.HID
54     WHERE I.PID = NEW.PID AND (S.UnitPrice IS NULL OR
55         S.UnitPrice <= 0);
56
57     IF missing_price > 0 THEN
58         SIGNAL SQLSTATE '45000'
59         SET MESSAGE_TEXT = 'Error: Cannot compute expense, medication price is missing or invalid in stock.';
60     END IF;
61
62     SELECT SUM(S.UnitPrice ) INTO new_total
63     FROM Includes I
64     JOIN Prescription PR ON I.PID = PR.PID
65     JOIN ClinicalActivity CA ON PR.CAID = CA.CAID
66     JOIN Department D ON CA.DEP_ID = D.DEP_ID
67     JOIN Stock S ON S.MID = I.MID AND S.HID = D.HID
68     WHERE I.PID = NEW.PID;
69
70     UPDATE Expense E
71     JOIN Prescription PR ON E.CAID = PR.CAID
72     SET E.Total = IFNULL(new_total, 0)
73     WHERE PR.PID = NEW.PID;
74
75 END//
76 DELIMITER ;
77
78 DELIMITER //
79
80 CREATE TRIGGER RecomputeExpenseAfterDelete
81 AFTER DELETE ON Includes
82 FOR EACH ROW
83 BEGIN
84
85     DECLARE new_total DECIMAL(10,2);
86
87     SELECT SUM(S.UnitPrice) INTO new_total
88     FROM Includes I
89     JOIN Prescription PR ON I.PID = PR.PID

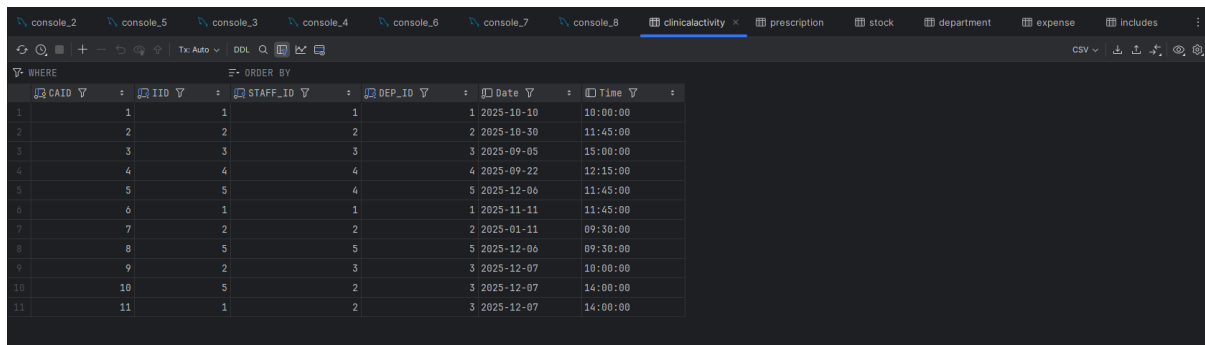
```

```

89      JOIN ClinicalActivity CA ON PR.CAID = CA.CAID
90      JOIN Department D ON CA.DEP_ID = D.DEP_ID
91      JOIN Stock S ON S.MID = I.MID AND S.HID = D.HID
92      WHERE I.PID = OLD.PID;
93
94      UPDATE Expense E
95      JOIN Prescription PR ON E.CAID = PR.CAID
96      SET E.Total = IFNULL(new_total, 0)
97      WHERE PR.PID = OLD.PID;
98  END//
99
100 DELIMITER ;
    
```

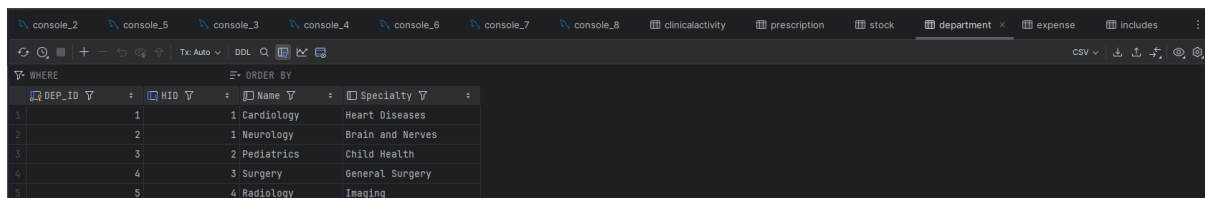
With the following tables data:

Clinical Activity:



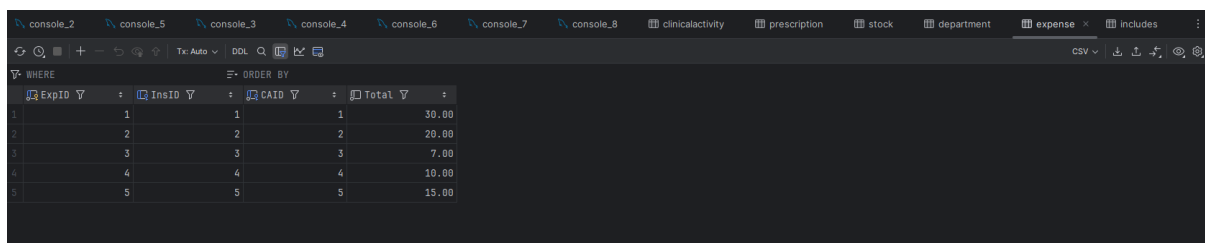
CAID	IID	STAFF_ID	DEP_ID	Date	Time
1	1	1	1	2025-10-10	10:00:00
2	2	2	2	2025-10-30	11:45:00
3	3	3	3	2025-09-05	15:00:00
4	4	4	4	2025-09-22	12:15:00
5	5	5	4	2025-12-06	11:45:00
6	6	1	1	2025-11-11	11:45:00
7	7	2	2	2025-01-11	09:30:00
8	8	5	5	2025-12-06	09:30:00
9	9	2	3	2025-12-07	10:00:00
10	10	5	2	2025-12-07	14:00:00
11	11	1	2	2025-12-07	14:00:00

Department:



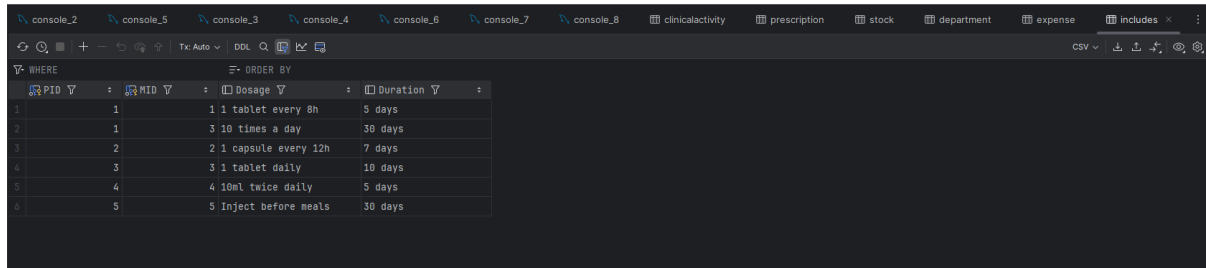
DEP_ID	HID	Name	Specialty
1	1	Cardiology	Heart Diseases
2	2	Neurology	Brain and Nerves
3	3	Pediatrics	Child Health
4	3	Surgery	General Surgery
5	4	Radiology	Imaging

Expense:



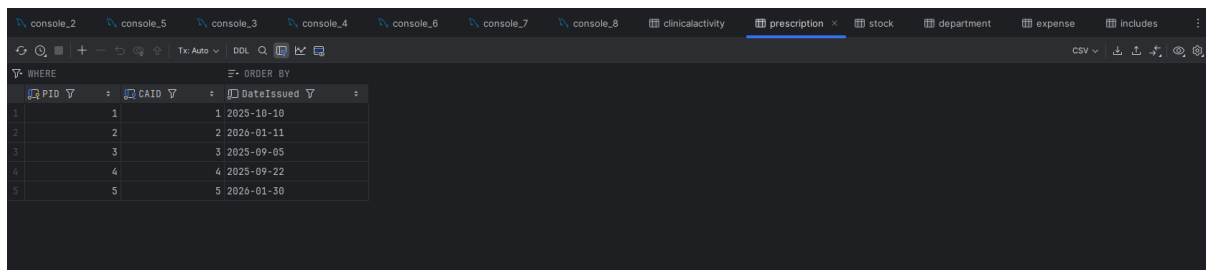
ExpID	InsID	CAID	Total
1	1	1	30.00
2	2	2	20.00
3	3	3	7.00
4	4	4	10.00
5	5	5	15.00

Includes:



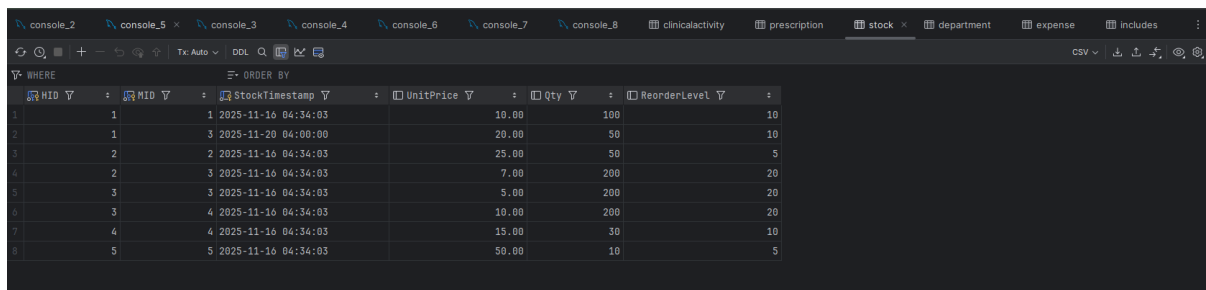
PID	MID	Dosage	Duration
1	1	1 tablet every 8h	5 days
2	1	3 10 times a day	30 days
3	2	2 1 capsule every 12h	7 days
4	3	3 1 tablet daily	10 days
5	4	4 10ml twice daily	5 days
6	5	5 Inject before meals	30 days

Prescription:



PID	CAID	DateIssued
1	1	2025-10-10
2	2	2026-01-11
3	3	2025-09-05
4	4	2025-09-22
5	5	2026-01-30

Stock:

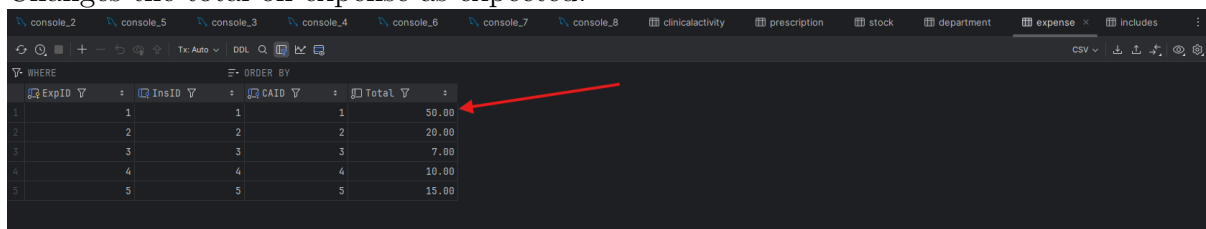


MID	StockTimestamp	UnitPrice	Qty	ReorderLevel
1	2025-11-16 04:34:03	10.00	100	10
1	2025-11-20 04:00:00	20.00	50	10
2	2025-11-16 04:34:03	25.00	50	5
2	2025-11-16 04:34:03	7.00	200	20
3	2025-11-16 04:34:03	5.00	200	20
3	2025-11-16 04:34:03	10.00	200	20
4	2025-11-16 04:34:03	15.00	30	10
5	2025-11-16 04:34:03	50.00	10	5

Now trying to run:

```
1 INSERT INTO includes VALUES
2 (1, 2, '3 times a day', '30 Days');
```

Changes the total on expense as expected:

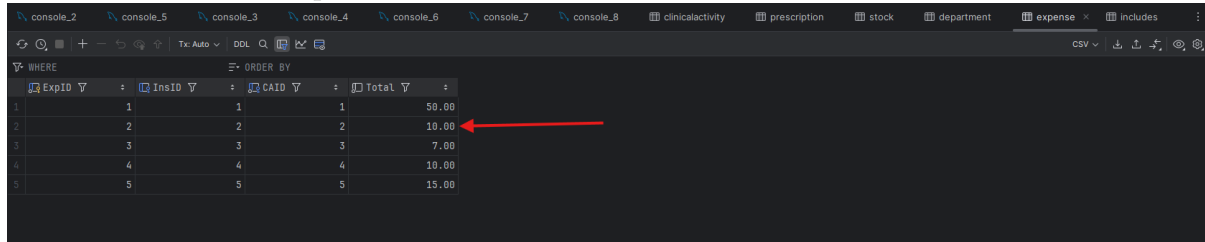


ExpID	InsID	CAID	Total
1	1	1	50.00
2	2	2	20.00
3	3	3	7.00
4	4	4	10.00
5	5	5	15.00

Now let's try changing up values:

```
1 UPDATE includes SET mid = 1 where pid = 2;
```

Here's the modified expense total:

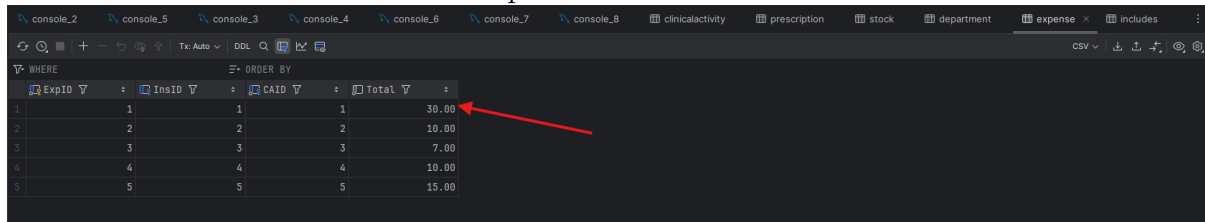


ExpID	InsID	CAID	Total
1	1	1	50.00
2	2	2	10.00
3	3	3	7.00
4	4	4	10.00
5	5	5	15.00

Now deleting a medication of a prescription:

```
1 DELETE FROM includes WHERE PID = 1 and MID = 3;
```

The amount is deducted from the expense total:



ExpID	InsID	CAID	Total
1	1	1	30.00
2	2	2	10.00
3	3	3	7.00
4	4	4	10.00
5	5	5	15.00

Trigger 3:

```
1 DELIMITER $$
2 CREATE TRIGGER PreventNegativeInconsistentStock_INSERT
3 BEFORE INSERT ON Stock
4 FOR EACH ROW
5 BEGIN
6     IF NEW.Qty<0 then
7         SIGNAL SQLSTATE '45000'
8         SET MESSAGE_TEXT = 'ERRR: Quantity cannot be Qty<0 or
9         any change decreasing Qty cannot drop below zero';
10    END IF;
11    IF NEW.UnitPrice<=0 then
12        SIGNAL SQLSTATE '45000'
13        SET MESSAGE_TEXT = 'ERRR: UnitPrice cannot be <-0';
14    END IF;
15    IF NEW.ReorderLevel < 0 then
16        SIGNAL SQLSTATE '45000'
17        SET MESSAGE_TEXT = 'ERRR: ReorderLevel cannot be <0';
18    END IF;
19 END$$
20 DELIMITER ;
21
22 DELIMITER $$
23 CREATE TRIGGER PreventNegativeInconsistentStock_UPDATE
24 BEFORE UPDATE ON Stock
25 FOR EACH ROW
26 BEGIN
27     IF NEW.Qty<0 then
```

```

29      SIGNAL SQLSTATE '45000'
30      SET MESSAGE_TEXT = 'ERRR: Quantity cannot be Qty<0 or
      any change decreasing Qty cannot drop below zero';
31  END IF;
32  IF NEW.UnitPrice <= 0 then
33      SIGNAL SQLSTATE '45000'
34      SET MESSAGE_TEXT = 'ERRR: UnitPrice cannot be <-0';
35  END IF;
36  IF NEW.ReorderLevel < 0 then
37      SIGNAL SQLSTATE '45000'
38      SET MESSAGE_TEXT = 'ERRR: ReorderLevel cannot be <0';
39  END IF;
40  END$$
41  DELIMITER ;

```

Now let's test if it works:

```

1  INSERT INTO Stock (HID, MID, Qty, UnitPrice, ReorderLevel)
2  VALUES (1, 5, -10, 15, 3);

```

The results as expected:



Now let's try:

```

1  INSERT INTO Stock (HID, MID, Qty, UnitPrice, ReorderLevel)
2  VALUES (1, 5, 10, 0, 3);

```



And:

```

1  INSERT INTO Stock (HID, MID, Qty, UnitPrice, ReorderLevel)
2  VALUES (1, 5, 10, 20, -1);

```



What about updating instead of inserting?

```

1  UPDATE Stock
2  SET Qty = -5
3  WHERE HID = 1 AND MID = 3;

```



```

1 UPDATE Stock
2 SET ReorderLevel = -2
3 WHERE HID = 1 AND MID = 3;

```



Trigger 4:

```

1 DELIMITER //
2 CREATE TRIGGER PreventPatientDelete
3 BEFORE DELETE ON Patient
4 FOR EACH ROW
5 BEGIN
6     DECLARE activity_count INT;
7     SELECT COUNT(*) INTO activity_count
8         FROM ClinicalActivity
9         WHERE IID = OLD.IID;
10    IF activity_count > 0 THEN
11        SIGNAL SQLSTATE '45000'
12        SET MESSAGE_TEXT = 'Error: Cannot delete patient with
13                               existing clinical activities. Please reassign or delete
14                               dependent activities first.';
15    END IF;
16 END//
17 DELIMITER ;

```

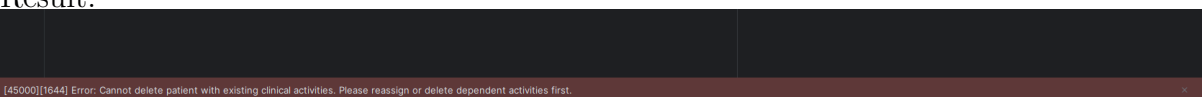
Now let's try creating a patient and a clinical activity and trying to delete the patient:

```

1 INSERT INTO Patient (IID, CIN, FullName, Birth, Phone)
2     VALUES (50, 'C123456', 'John Doe', '1985-03-12',
3             '0612345678');
4 INSERT INTO ClinicalActivity (CAID, IID, STAFF_ID, DEP_ID, Date,
5                               Time)
6     VALUES (200, 50, 4, 3, '2025-01-12', '09:00:00');
7 DELETE FROM Patient WHERE IID = 50;

```

Result:



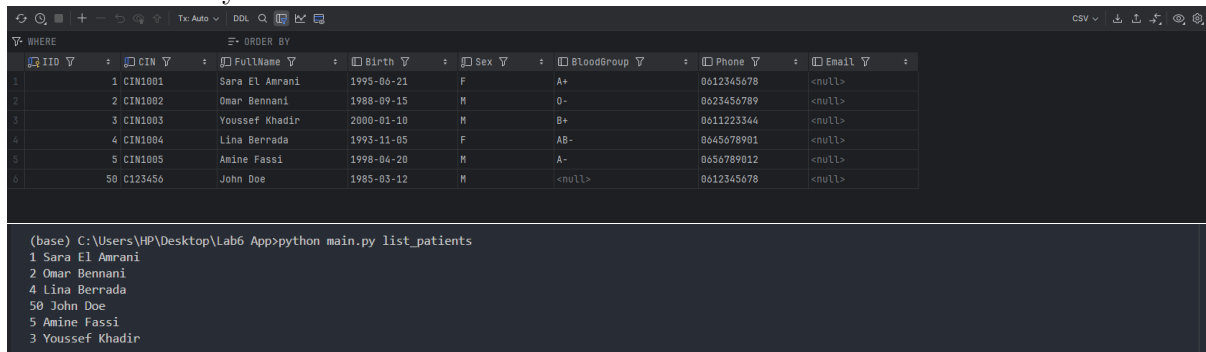
App Development:

How to use:

The folder containing the python files is provided in the repository.

The usage of the app is fairly simple, make sure the files are kept within the same folder. Then open up cmd to the folder's path, here's the list of the commands that are available as well as the arguments alongside them:

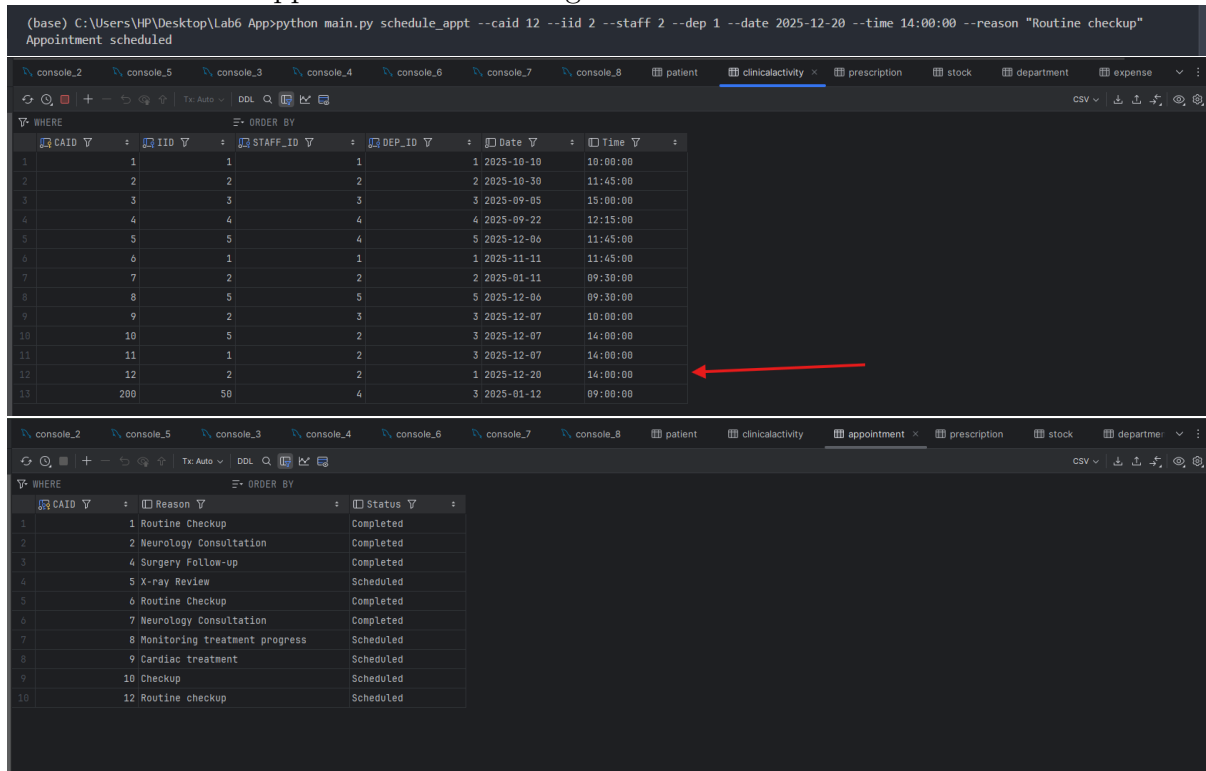
`python main.py list_patients` (no arguments) It's supposed to show the first 20 patients ordered by fullname:



```
(base) C:\Users\HP\Desktop\Lab6 App>python main.py list_patients
1 Sara El Amrani
2 Omar Bennani
4 Lina Berrada
50 John Doe
5 Amine Fassi
3 Youssef Khadir
```

`python main.py schedule_appt --caid ... --iid ... --staff ... --dep ... --date ... --time ... --reason ...`

Used to create an appointment with the given data:



```
(base) C:\Users\HP\Desktop\Lab6 App>python main.py schedule_appt --caid 12 --iid 2 --staff 2 --dep 1 --date 2025-12-20 --time 14:00:00 --reason "Routine checkup"
Appointment scheduled
```

CAID	IID	STAFF_ID	DEP_ID	Date	Time
1	1	1	1	2025-10-10	10:00:00
2	2	2	2	2025-10-30	11:45:00
3	3	3	3	2025-09-05	15:00:00
4	4	4	4	2025-09-22	12:15:00
5	5	5	4	2025-12-06	11:45:00
6	6	1	1	2025-11-11	11:45:00
7	7	2	2	2025-01-11	09:30:00
8	8	5	5	2025-12-06	09:30:00
9	9	2	3	2025-12-07	10:00:00
10	10	5	2	2025-12-07	14:00:00
11	11	1	2	2025-12-07	14:00:00
12	12	2	2	2025-12-20	14:00:00
13	200	50	4	2025-01-12	09:00:00

`python main.py low_stock` (no arguments) To show all the medications per hospital that are below reorderlevel or the medications that are not within that hospital:

The screenshot shows a database application interface with two tables displayed.

Table 1: stock

ID	HID	StockTimestamp	UnitPrice	Qty	ReorderLevel
1	1	2025-11-10 04:34:03	10.00	100	10
2	1	2025-12-20 13:00:00	20.00	100	5
3	1	2025-11-20 04:00:00	20.00	3	10
4	2	2025-11-10 04:34:03	25.00	50	5
5	2	2025-11-10 04:34:03	7.00	200	20
6	3	2025-11-10 04:34:03	5.00	200	20
7	3	2025-11-10 04:34:03	10.00	200	20
8	4	2025-11-10 04:34:03	15.00	30	10
9	4	2025-11-10 04:00:00	15.00	50	10
10	5	2025-11-10 04:34:03	50.00	10	5

Table 2: medication

ID	Name	Form	Strength	ActiveIngredient	TherapeuticClass	Manufacturer
1	Paracetamol	Tablet	500mg	Paracetamol	Analgesic	Sanofi
2	Amoxicillin	Capsule	250mg	Amoxicillin	Antibiotic	Pfizer
3	Aspirin	Tablet	100mg	Acetylsalicylic Acid	Analgesic	Bayer
4	Cough Syrup	Liquid	5mg/10mL	Dextromethorphan	Antitussive	BSK
5	Insulin	Injection	10IU/mL	Insulin	Antidiabetic	Novo Nordisk

Terminal Output:

```
(base) C:\Users\HP\Desktop\Lab6 App>python main.py low_stock
Amoxicillin      | CHU Agadir      | Stock: None (reorder: None)
Aspirin          | CHU Agadir      | Stock: None (reorder: None)
Cough Syrup      | CHU Agadir      | Stock: None (reorder: None)
Paracetamol      | CHU Agadir      | Stock: None (reorder: None)
Cough Syrup      | CHU Casablanca  | Stock: None (reorder: None)
Insulin          | CHU Casablanca  | Stock: None (reorder: None)
Paracetamol      | CHU Casablanca  | Stock: None (reorder: None)
Amoxicillin      | CHU Fès         | Stock: None (reorder: None)
Aspirin          | CHU Fès         | Stock: None (reorder: None)
Paracetamol      | CHU Fès         | Stock: None (reorder: None)
Amoxicillin      | CHU Marrakech   | Stock: None (reorder: None)
Insulin          | CHU Marrakech   | Stock: None (reorder: None)
Paracetamol      | CHU Marrakech   | Stock: None (reorder: None)
Aspirin          | CHU Rabat       | Stock: 3 (reorder: 10)
Cough Syrup      | CHU Rabat       | Stock: None (reorder: None)
Insulin          | CHU Rabat       | Stock: None (reorder: None)
```

`python main.py staff_share` (no arguments) This command shows the percentage of each staff contribution to the appointments:

The screenshot shows a database application interface with two tables displayed.

Table 1: clinicalactivity

CAID	HID	STAFF_ID	DEP_ID	Date	Time
1	1	1	1	2025-10-10	10:00:00
2	2	2	2	2025-10-30	11:45:00
3	3	3	3	2025-09-05	15:00:00
4	4	4	4	2025-09-22	12:15:00
5	5	5	4	2025-12-06	11:45:00
6	6	1	1	2025-11-11	11:45:00
7	7	2	2	2025-01-11	09:30:00
8	8	5	5	2025-12-06	09:50:00
9	9	2	3	2025-12-07	10:00:00
10	10	5	2	2025-12-07	14:00:00
11	11	1	2	2025-12-07	14:00:00
12	12	2	2	2025-12-20	14:00:00
13	200	50	4	2025-01-12	09:00:00

Table 2: department

DEP_ID	HID	Name	Specialty
1	1	Cardiology	Heart Diseases
2	1	Neurology	Brain and Nerves
3	2	Pediatrics	Child Health
4	3	Surgery	General Surgery
5	4	Radiology	Imaging

```
(base) C:\Users\HP\Desktop\Lab6 App>python main.py staff_share
Staff ID   Staff Name           TotalAppointments PercentageShare %
2          Dr. Salma Tazi        3                60.00000
1          Dr. Hicham El Idrissi 2                40.00000
4          Technician Yassine Kabbaj 1              100.00000
4          Technician Yassine Kabbaj 1                50.00000
5          Dr. Mehdi Rahali      1                50.00000
3          Nurse Rania Lahlou    1                50.00000
2          Dr. Salma Tazi        1                50.00000
```

5 Discussion

Challenges faced, observations, lessons learned.

Challenges Faced:

Throughout this lab, several challenges were encountered across the three main parts: building views, implementing triggers, and developing the application layer.

5.0.1 Designing and Implementing Views:

Understanding the schema and determining the correct join paths took time, especially for views such as `UpcomingByHospital` and `PatientNextVisit`, which involved filtering by dates and selecting only specific appointment statuses. Another challenge was ensuring that the views remained efficient and correctly grouping aggregated information. For the `DrugPricingSummary` view, we initially struggled to understand what the grouping really required. At first, we thought the task meant calculating the statistics for the same medication across different hospitals, while another interpretation was to compute the summary only inside each hospital. This caused confusion because both seemed possible.

5.0.2 Building Triggers:

During the development of the required triggers for the MNHS database, several challenges were encountered. The first difficulty was handling tables that follow a parent-child structure, such as `ClinicalActivity` and `Appointment`. Since the `Appointment` table does not contain scheduling attributes (`Date`, `Time`, `STAFF_ID`), the trigger could not directly access these values using `NEW`. This required retrieving the corresponding data from the `ClinicalActivity` table using declared variables and a `SELECT ... INTO` statement, which added complexity to the trigger logic. The trigger responsible for recomputing `Expense.Total` after `INSERT`, `UPDATE`, or `DELETE` operations on the `Includes` table was the most complex one. It required navigating through several related tables: `Includes` → `Prescription` → `ClinicalActivity` → `Expense`, and joining with `Stock` through hospital and medication information. Besides, writing conditions correctly and ensuring that meaningful error messages were raised with `SIGNAL` was a key challenge.

5.0.3 Building the Application layer:

Since this was our first time building an application that interacts directly with a database, a significant part of the challenge was simply understanding the new concepts involved. We had to learn how SQL and Python communicate with each other, how connections are established, and how queries are executed from within Python code. Much of our time was spent understanding how to correctly use database connectors, environment variables, and parameterized queries. Another difficulty was making the connection between the logic implemented in SQL such as triggers, constraints, transaction and how Python should handle these rules when performing inserts or updates.

5.0.4 Debugging and Testing

Testing the views, triggers, and application code was one of the most time-consuming challenges which required patience and careful observation since we had to try many

different cases, verify outputs carefully, and make sure all views, triggers, and application functions behaved correctly in every scenario.

Observations:

Lab 6 demonstrated how triggers, views, and the application layer work together to enforce data integrity and simplify system development. Triggers ensured that essential business rules, such as preventing double booking, maintaining accurate expenses, validating stock values, and blocking patient deletion when activities exist. Views helped reduce query complexity by encapsulating repeated joins and presenting clean, high-level representations of the data for easier access. The application layer illustrated secure and efficient backend design using environment variables, parameterized queries, and transactions, relying on the database's triggers and views to guarantee correctness. Overall, the lab showed that distributing responsibilities between the database and the application leads to a more reliable, maintainable, and scalable system.

Lessons Learned:

This laboratory provided extensive and practical lessons in transforming a basic database into a robust, intelligent system that actively supports real-world applications. We learned that views function as essential simplification tools, packaging intricate database relationships and calculations into straightforward, ready-to-use data summaries. By hiding the underlying complexity of joins and filters, views save significant development time, reduce errors, and ensure that every part of an application uses the same consistent logic for reporting and data display.

Our work with triggers revealed the power of building business rules directly into the database's foundation. These automated guardians work constantly behind the scenes to enforce critical policies like preventing staff double-booking, ensuring medication stock levels never go negative, maintaining accurate financial totals, and protecting patient records from accidental deletion. This approach guarantees data integrity and business rule compliance at all times, regardless of which application or user interface accesses the database.

Furthermore, we gained crucial experience in bridging the gap between database and application. We implemented secure connection methods, managed transactions to ensure data consistency during multi-step operations, and designed command-line tools that make complex data accessible for everyday tasks like checking low stock or scheduling appointments. This practice highlighted the importance of thoughtful error handling and user-friendly output formatting. Ultimately, this project demonstrated that a well-executed database is far more than a passive storage container. It becomes an active, collaborative partner in the software ecosystem. By embedding intelligence, rules, and ready-to-use data perspectives directly into the database layer, we create systems that are more reliable, easier to maintain, and more adaptable to future needs, forming a solid foundation for any application built upon it.

6 Conclusion

In this part of the project, we designed and implemented four views to simplify complex joins and improve both the usability and performance of key MNHS queries. We also developed multiple triggers to enforce essential business rules, ensure data consistency, and protect the integrity of the MNHS database. Finally, we built a Python-based application layer on top of the MNHS schema.